Lab 07:

Polymorphism in Java

(2 sessions to complete)

Questions:

a. **Create your project as a Maven project.**
b. **Create all the classes needed to support this system.**
c. **Use object-oriented principles accordingly: inheritance, polymorphism.**
d. **Maximize code reuse.**
e. **Create a main program in your main method to test your classes.**
f. **Integrate a file logger to log system start and stop events and raised exceptions when needed in the main calling program.**

Scenario:

A travel agency wants to build a system to manage **different types of transportation bookings** for its customers. The agency offers three transportation modes:

- **Flights**
- **Train rides**
- **Car rentals**

All bookings share some common information:

- A **booking ID**
- The **customer's full name**
- The **date of travel**
- The **destination city**

Each booking type has its own additional data and its own rules for computing the **total price**, which is why the agency wants to use **polymorphism**.

Booking Types & Their Rules

1. FlightBooking

- Stores:
  - Base ticket price (set when booking is created)
  - Luggage weight (entered later by the customer)
- Price calculation:
  - Total price = base price + (luggage weight × extra-luggage rate)
- Business rules:
  - Luggage weight must be between **0 and 40 kg**
  - If luggage weight is not provided → throw `MissingInformationException`
  - If weight is outside allowed range → throw `InvalidArgumentException`

- The extra-luggage rate is defined in a **global configuration class**.

2. TrainBooking

- Stores:
  - Seat class (STANDARD, FIRST_CLASS)
  - Distance in kilometers (entered later by the system)
- Price calculation:
  - STANDARD: price = distance × standard rate
  - FIRST_CLASS: price = distance × first-class rate
- Business rules:
  - Distance must be between **1 and 2000 km**
  - If distance is not provided → throw `MissingInformationException`
  - If distance is invalid → throw `InvalidArgumentException`
- Rates are defined in the **global configuration class**.

3. CarRentalBooking

- Stores:
  - Daily rental rate (set when booking is created)
  - Number of rental days (entered later by the customer)
- Price calculation:
  - Total price = daily rate × number of days
- Business rules:
- Number of days must be between **1 and 30**
- If days are not provided → throw `MissingInformationException`
- If days are invalid → throw `InvalidArgumentException`

Additional Requirements

Global Configuration Class

Contains constants such as:

- EXTRA_LUGGAGE_RATE
- TRAIN_STANDARD_RATE
- TRAIN_FIRST_CLASS_RATE
- MAX_LUGGAGE_WEIGHT
- MAX_TRAIN_DISTANCE
- MAX_RENTAL_DAYS
- etc.

Custom Exceptions

- `MissingInformationException`
- `InvalidArgumentException`

Polymorphism Requirement

A method like: **computeTotalPrice**(Booking booking) should work for **any** booking type.

Annex :

(run maven build with goal: clean package exec:java)

1- Configure the exec maven plugin and SLF4j with log4j12:

```
<dependencies>

        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>2.0.16</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-log4j12</artifactId>
            <version>2.0.16</version>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.6.3</version>
                <configuration>
                  <mainClass>edu.spu.se411.lab06_logging.App</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
```

2- Then configure log4j12 by adding log4j.properties in src/main/resources

```
log4j.rootLogger = INFO, LOG1
log4j.appender.LOG1=org.apache.log4j.FileAppender
log = ./logs/App/log4j
log4j.appender.LOG1.File=${log}/log.out
log4j.appender.LOG1.layout=org.apache.log4j.PatternLayout
log4j.appender.LOG1.layout.conversionPattern=%d %p [%t] %C - %m%n
```

3- Logging: import Logger then add logging instructions

```
static Logger logger = LoggerFactory.getLogger(App.class);
```

Example: *logger*.info("Application is starting...");